COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 07/17/2026 | **DATE SUBMITTED:** | 07/22/2026 |
| **NAME:** | Juan Paulo C. Lara | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 2.1** **Arrays, Pointers and Dynamic Memory Allocation**

## 1. PROCEDURE OUTPUT

What procedures did you perform?

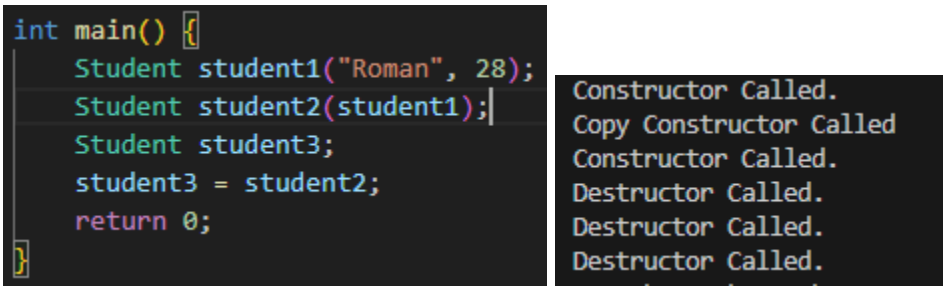
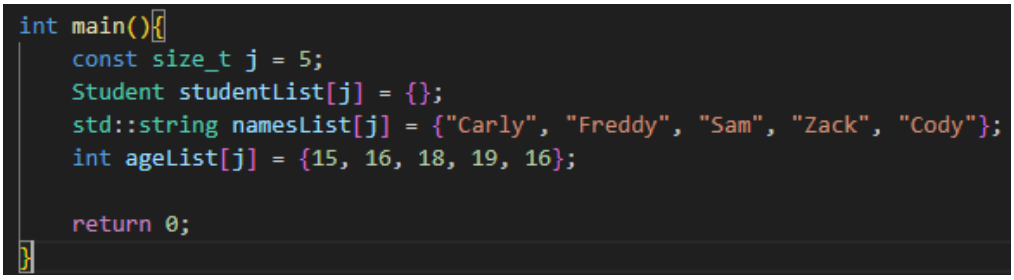
| | |
|---|---|
| Screenshot |  |
| Observation | As seen in this table, the function student1 calls the constructor to initialize and set the values of variables student and age, and then prints “Constructor Called” to prove the constructor has executed. Next, the function student2 calls the copy constructor to print a string to confirm its execution, and then it copies the data from student1 to student 2. The variable student3 calls the constructor to do a similar process but assigns the default values from the constructor instead. Then, the last line sets student3 to student2, which then reassigns the addresses of student3 to student2 using the ‘this ->’ pointer. And, when return 0 is executed, all the three lines executed before call the destructor to uninitialize them. |

Table 2-1. Initial Driver Program

| | |
|---|---|
| Screenshot |  |

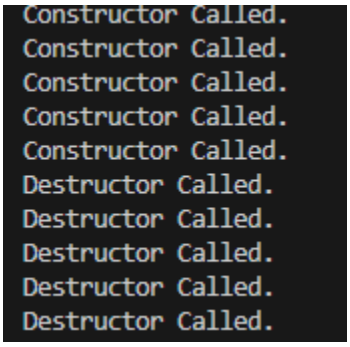
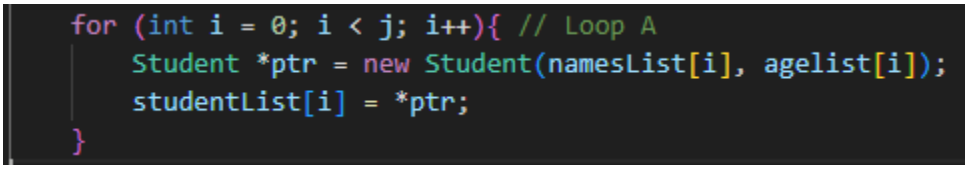
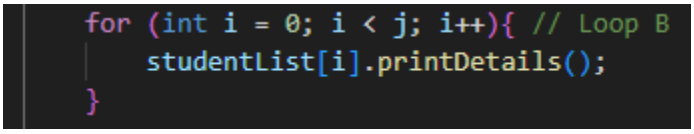
| | |
|---|---|
| |  |
| Observation | As seen in this table, the student list array calls the constructor to initialize the default name and age, and add them into the list. Because j is set to 5, the process is done repeatedly for five times with the assignment of student names and ages being the last call. Next, the array of student names and ages allocate the array with their assigned elements with their respective data. After this, when return 0 is executed, the destructor uninitializes the data of the lists. |

Table 2-2. Modified Driver Program with Student Lists

| | |
|---|---|
| Loop A |  |
| Observation | Inside Loop A, the pointer assigns a new list of students by assigning the addresses of the strings into a new pointer list. Another step counts the variable i up to five to store the pointer data into the list. This is a process of converting the data filled with strings into addresses, which the next for loop uses its pointer to print back the original strings. |
| Loop B |  |
| Observation | Inside Loop B, after the execution of Loop A, the lists are printed by calling the pintDetails function to display the values/strings inside the student list array. This uses the pointer in the function to convert back the pointer into the original strings. Since this uses a for loop, the process is done five times to fill up the five elements in the array. |

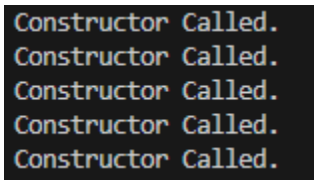
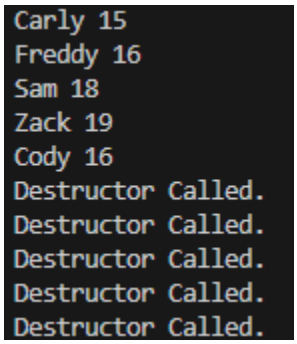
| | |
|---|---|
| Output |  <br>Loop A<br>  <br>Loop B |
| Observation | Noticing the pattern, Loop A calls the constructor function to assign the strings and converts them into addresses for the pointer function to print later. Loop B points back to the addresses of the strings and displays their original names and ages of the students. After this process, the destructor is called to clear up the five items allocated in memory. |

Table 2-3. Final Driver Program

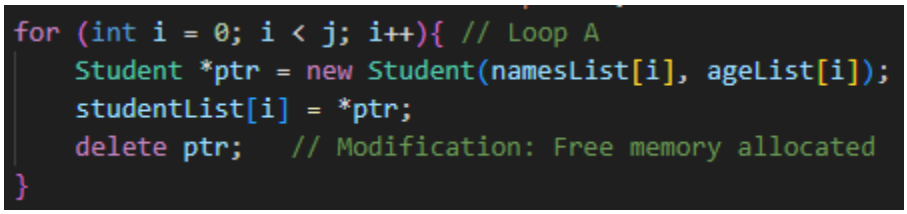
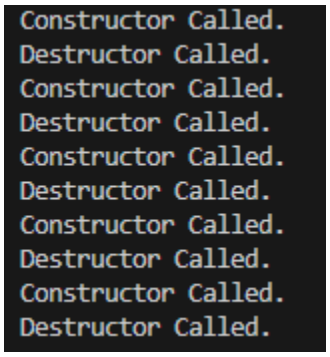
| | |
|---|---|
| Screenshot |  <br>  |
| Observation | This modification in the code adds an instruction to delete the pointer to clear up the allocated memory addresses in the program's heap. When the constructor has been called, the destructor comes next to clear up the data once allocated by the elements in the list. This is done to prevent memory leakage and keep the memory space clean. |

Table 2-4. Modifications/Corrections Necessary

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
class Fruit{
private:
    std::string name;
    double price;
    int quantity;

public:
    // Constructor
    Fruit(std::string newName = "", double newPrice = 0, int newQuantity = 0){
        name = std::move(newName);
        price = newPrice;
        quantity = newQuantity;
    }

    // Destructor
    ~Fruit(){
        std::cout << "Fruit Destructor Called\n";
    }

    // Copy Constructor
    Fruit(const Fruit &copyFruit){
        std::cout << "Fruit Cropy Constructor Called\n";
        name = copyFruit.name;
        price = copyFruit.price;
        quantity = copyFruit.quantity;
    }

    double getTotalPrice(){
        return price * quantity;
    }

    void printDetails(){
        std::cout << "Fruit name: " << name << "\nFruit Price: P" << price << "\nFruit Quantity: " << quantity << "\nFruit Total Price: P" <<
        getTotalPrice() << "\n\n";
    }

};
```

```cpp
class Vegetable{
private:
    std::string name;
    double price;
    int quantity;

public:
    // Consttuctor
    Vegetable(std::string newName = "", double newPrice = 0, int newQuantity = 0){
        name = std::move(newName);
        price = newPrice;
        quantity = newQuantity;
    }

    // Destructor
    ~Vegetable(){
        std::cout << "Vegetable Destructor Called\n";
    }

    // Copy Constructor
    Vegetable(const Vegetable &copyVegetable){
        std::cout << "Vegetable Copy Constructor Called\n";
        name = copyVegetable.name;
        price = copyVegetable.price;
        quantity = copyVegetable.quantity;
    }

    double getTotalPrice(){
        return price * quantity;
    }

    void printDetails(){
        std::cout << "Vegetable name: " << name << "\nVegetable Price: P" << price << "\nVegetable Quantity: " << quantity << "\nVegetable Total
        Price: P" << getTotalPrice() << "\n\n";
    }
};
```

Figure 1. Fruit and Vegetable Classes

These classes for fruit and vegetable contain private and public attributes, which contain the three important parts of these classes, which are constructor, destructor, and copy constructor. These three elements allow for properly allocating and writing parts of the memory, which get more important as programs become more complex and allocate more memory spaces. By creating these classes with these three functions, one can have an easier time understanding the aspects of dynamic memory allocation with pointers and arrays, and applying these concepts to larger programs.

```
104     // Fruits & Vegetables
105     fruitList[0] = new Fruit("Apple", 10, 7);
106     fruitList[1] = new Fruit("Banana", 10, 8);
107     vegetableList[0] = new Vegetable("Broccoli", 60, 12);
108     vegetableList[1] = new Vegetable("Lettuce", 50, 10);
```

Figure 2. Items Set in the Grocery List.

These objects are dynamically allocated to the lists of the fruits and vegetables with their own set of values assigned to each object. Each fruit and vegetable object contains a format of the parameters set, which are the name, price, and quantity, and these are used for displaying and computing the values to get the total price. These objects allocate their memory address space, where all the values are stored.

```
77  double TotalSum(Fruit *fruitList[], size_t fruitSize, Vegetable *vegetableList[], size_t vegetableSize){
78      double total = 0;
79      // Add the total price of each fruit and vegetable total
80      for (size_t i = 0; i < fruitSize; i++){
81          if (fruitList[i] != nullptr){
82              total += fruitList[i]->getTotalPrice();
83          }
84      }
85
86      for (size_t i = 0; i < vegetableSize; i++){
87          if (vegetableList[i] != nullptr){
88              total += vegetableList[i]->getTotalPrice();
89          }
90      }
91
92      return total;
93  }
```

Figure 3. Function to get the Sum of All Items.

This function is where the process of computing the total sum of all the items takes place. Before the total begins proceeding to iterating values based on the set array sizes, it first resets the total value to zero. This step is important because it prevents the chances of getting incorrect values when the program is run multiple times. Then, these two for loop blocks contain the addition process for adding the total price of the fruits and vegetables to the total variable. After completing this process, the total value is returned for output.

```
124     // Delete the Lettuce
125     delete vegetableList[1];
126     vegetableList[1] = nullptr;
127     std::cout << "Lettuce has been deleted from the list\n";
```

Figure 4. Deleting the Lettuce.

To delete the lettuce, the vegetableList variable with the specified index is set to the delete instruction to remove

this specified index from the list to de-allocate the assigned memory space for the element. This instruction renders the address as free and no longer assigned to any variable, object, or class but the address value still remains. So, to fully de-allocate and completely free this index, a null pointer is placed to ensure the value it points to is nothing and nowhere near any address space that stray pointers could see, which can prevent memory leaking, unhandled exceptions, and crashes at the compiled program's runtime.

| Conclusion |
| --- |
| This hands-on activity has demonstrated the important aspects of arrays, pointers, and dynamic memory allocation through the use of classes, objects, and following the rule of three. The rule of three is composed of destructors, copy assignments operators, and copy assignment operators, which serve as the basis of memory management, allocation and de-allocation, and copying and transferring data across the program heap. The procedure demonstrated the use of the rule of three by using the basic constructor, destructor, and copy constructor to create a default set of values, remove and delete values, and create a copy of another data. The supplementary activities tested the more complex parts of implementing dynamic memory allocation using a real-world problem of finding the sum of a type of item, the total of all items, and the list of items with their attributes and values shown. This gave many challenges in finding ways to properly allocate objects, calculate values, and display them while following the given instructions as closely as possible, such as solving a compiler error from a stray pointer. Overall, this activity has helped create a better understanding of memory allocation, pointers, and how they behave in the memory space. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Google Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Clarification of Concepts, correction in writing typos, and code assistance**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **The AI was used in clarifying concepts in the discussion, providing basic tips for proper formatting and assigning values in classes.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.




Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.